BFS, DFS and IDDFS

AIM

To implement and analyze BFS, DFS, and IDDFS search algorithms to explore a graph or tree and understand their traversal strategies.

Description

BFS explores nodes level by level using a queue. DFS explores as deeply as possible before backtracking using a stack or recursion. IDDFS combines DFS's memory efficiency with BFS's completeness by repeatedly performing depth-limited searches with increasing depth limits.

BFS

Code

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

start = 'A'
goal = 'G'

open_list = deque([start])
closed_list = []
parent = {start: None}

while open_list:
    node = open_list.popleft()

    if node in closed_list:
```

```
        continue

    closed_list.append(node)

    print("OPEN:", list(open_list))
    print("CLOSED:", closed_list)

    if node == goal:
        break

    for neighbour in graph[node]:
        if neighbour not in closed_list and neighbour not in open_list:
            parent[neighbour] = node
            open_list.append(neighbour)

path = []

if goal in closed_list:
    node = goal

    while node is not None:
        path.append(node)
        node = parent[node]

    path.reverse()

print("\nPath:", " -> ".join(path))
print("Final OPEN:", list(open_list))
print("Final CLOSED:", closed_list)
```

Output

```
OPEN: []
CLOSED: ['A']
OPEN: ['C']
CLOSED: ['A', 'B']
OPEN: ['D', 'E']
CLOSED: ['A', 'B', 'C']
OPEN: ['E', 'F', 'G']
CLOSED: ['A', 'B', 'C', 'D']
OPEN: ['F', 'G']
CLOSED: ['A', 'B', 'C', 'D', 'E']
OPEN: ['G']
CLOSED: ['A', 'B', 'C', 'D', 'E', 'F']
OPEN: []
```

```
CLOSED: ['A', 'B', 'C', 'D', 'E', 'F', 'G']

Path: A -> C -> G
Final OPEN: []
Final CLOSED: ['A', 'B', 'C', 'D', 'E', 'F', 'G']
```

DFS

Code

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

start = 'A'
goal = 'G'

open_list = [start]
closed_list = []
parent = {start: None}

while open_list:
    node = open_list.pop()

    if node in closed_list:
        continue

    closed_list.append(node)

    print("OPEN:", open_list)
    print("CLOSED:", closed_list)

    if node == goal:
        break

    for neighbour in reversed(graph[node]):
        if neighbour not in closed_list and neighbour not in open_list:
            parent[neighbour] = node
```

```
            open_list.append(neighbour)

path = []

if goal in closed_list:
    node = goal

    while node is not None:
        path.append(node)
        node = parent[node]

    path.reverse()

print("\nPath:", " -> ".join(path))
print("Final OPEN:", open_list)
print("Final CLOSED:", closed_list)
```

Output

```
OPEN: []
CLOSED: ['A']
OPEN: ['C']
CLOSED: ['A', 'B']
OPEN: ['C', 'E']
CLOSED: ['A', 'B', 'D']
OPEN: ['C']
CLOSED: ['A', 'B', 'D', 'E']
OPEN: []
CLOSED: ['A', 'B', 'D', 'E', 'C']
OPEN: ['G']
CLOSED: ['A', 'B', 'D', 'E', 'C', 'F']
OPEN: []
CLOSED: ['A', 'B', 'D', 'E', 'C', 'F', 'G']

Path: A -> C -> G
Final OPEN: []
Final CLOSED: ['A', 'B', 'D', 'E', 'C', 'F', 'G']
```

IDDFS

Code

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

start = 'A'
goal = 'G'


def depth_limited_search(start, goal, limit):

    open_list = [(start, 0)]
    closed_list = []
    parent = {start: None}

    while open_list:

        node, depth = open_list.pop()

        if node in closed_list:
            continue

        closed_list.append(node)

        print("OPEN:", open_list)
        print("CLOSED:", closed_list)

        if node == goal:

            path = []
            current = node

            while current is not None:
                path.append(current)
                current = parent[current]

            path.reverse()

            return path, open_list, closed_list
```

```
        if depth < limit:

            for neighbour in reversed(graph[node]):

                if neighbour not in closed_list:
                    parent[neighbour] = node
                    open_list.append((neighbour, depth + 1))

    return None, open_list, closed_list


depth = 0

while True:

    print("\n==============================")
    print("DEPTH LIMIT:", depth)
    print("==============================")

    path, open_list, closed_list = depth_limited_search(
        start,
        goal,
        depth
    )

    if path is not None:
        print("\nGoal Found!")
        print("Path:", " -> ".join(path))
        print("Final OPEN:", open_list)
        print("Final CLOSED:", closed_list)
        break

    depth += 1
```

Output

```
==============================
DEPTH LIMIT: 0
==============================
OPEN: []
CLOSED: ['A']

==============================
DEPTH LIMIT: 1
==============================
```

```
OPEN: []
CLOSED: ['A']
OPEN: [('C', 1)]
CLOSED: ['A', 'B']
OPEN: []
CLOSED: ['A', 'B', 'C']

==============================
DEPTH LIMIT: 2
==============================
OPEN: []
CLOSED: ['A']
OPEN: [('C', 1)]
CLOSED: ['A', 'B']
OPEN: [('C', 1), ('E', 2)]
CLOSED: ['A', 'B', 'D']
OPEN: [('C', 1)]
CLOSED: ['A', 'B', 'D', 'E']
OPEN: []
CLOSED: ['A', 'B', 'D', 'E', 'C']
OPEN: [('G', 2)]
CLOSED: ['A', 'B', 'D', 'E', 'C', 'F']
OPEN: []
CLOSED: ['A', 'B', 'D', 'E', 'C', 'F', 'G']

Goal Found!
Path: A -> C -> G
Final OPEN: []
Final CLOSED: ['A', 'B', 'D', 'E', 'C', 'F', 'G']
```